

Báo cáo triển khai điều chế đồng bộ SOPWM

Trên vi điều khiển TI TMS320F280049C

VietNH | Dự án epwm_svm

June 28, 2026

Abstract

Báo cáo trình bày quy trình triển khai phương pháp *Selective Optimal Pulse Width Modulation* (SOPWM) — một dạng điều chế xung đồng bộ (synchronous PWM) — trên DSP C2000 TMS320F280049C. Ba thách thức kỹ thuật trung tâm được phân tích và giải quyết: (i) giới hạn phần cứng ePWM chỉ có hai thanh ghi so sánh CMPA/CMPB trong khi cần tới $4M$ sự kiện chuyển mạch ($M = 3 \dots 7$, tức 12–28 điểm) trên mỗi chu kỳ cơ bản; (ii) yêu cầu bắt buộc sử dụng DMA để cập nhật góc so sánh mỗi chu kỳ sóng mang với độ chính xác thời gian cao; (iii) phân bố bộ nhớ FLASH/RAM do khối lượng lớn bảng tra cứu góc (LUT) và bảng lịch runtime. Các công thức toán học, kiến trúc phần mềm và tham số triển khai thực tế được mô tả đầy đủ.

Contents

1	Giới thiệu	3
1.1	Bối cảnh: điều chế đồng bộ và SOPWM	3
1.2	Nền tảng phần cứng	3
2	Vấn đề 1: Giới hạn phần cứng — hai CMP nhưng cần $4M$ sự kiện	3
2.1	Phát biểu bài toán	3
2.2	Nguyên lý giải pháp: bảng lịch theo chu kỳ mang	4
2.3	Mở rộng quarter-wave từ LUT	4
2.4	Đối xứng nửa chu kỳ và ba pha	4
3	Vấn đề 2: Tính cần thiết của DMA trong cập nhật góc	5
3.1	Tại sao CPU không đủ	5
3.2	Cấu hình DMA trong dự án	5
3.3	Đồng bộ DMA – ISR – vòng lặp chính	6
3.4	Chỉ số điều chế và cập nhật góc	6
4	Vấn đề 3: Phân bố bộ nhớ FLASH/RAM	6
4.1	Phát biểu vấn đề	6
4.2	Cấu hình bộ nhớ trong dự án	8
4.2.1	Bản đồ vật lý F280049C (trích 28004x_generic_flash_lnk.cmd)	8

4.2.2	Cấu hình section trong file liên kết	8
4.2.3	Ràng buộc build CCS	8
4.3	Đóng góp của mã nguồn	8
4.3.1	Minh hoạ placement trong <code>sopwm.c</code>	10
4.4	Sơ đồ phân vùng logic	10
4.5	Chi tiết dung lượng LUT	10
4.6	Kết quả và hệ quả thiết kế	10
5	Công thức tổng hợp và ví dụ số	11
5.1	Ví dụ: $N = 7$, $m = 0,50$	11
5.2	Điều kiện tính đúng của SOPWM đồng bộ	11
6	Kết luận	12
A	Tham chiếu mã nguồn	12

1 Giới thiệu

1.1 Bối cảnh: điều chế đồng bộ và SOPWM

Trong điều khiển biến tần ba pha, **điều chế đồng bộ** (synchronous PWM) là phương pháp trong đó số xung trong một chu kỳ sóng cơ bản $T_1 = 1/f_1$ được cố định (hoặc chọn theo quy tắc rõ ràng), và các instant chuyển mạch được xác định *đồng bộ* với pha điện của điện áp tham chiếu. Khác với SVPWM hay carrier-based PWM bất đồng bộ, SOPWM tối ưu hóa trực tiếp các góc chuyển mạch α_k sao cho:

- loại bỏ các harmonic thấp tại bội số $N \pm 1$ của số xung N ;
- giữ biên độ điện áp cơ bản theo chỉ số điều chế m .

Với số xung lẻ $N \in \{7, 9, 11, 13, 15\}$, bài toán tối ưu chỉ cần giải trên **phần tư chu kỳ** (0° – 90°) nhờ đối xứng lẻ:

$$v(\theta) = -v(\theta + 180^\circ), \quad v(\theta) = v(-\theta). \quad (1)$$

Do đó chỉ cần lưu $M = \lfloor N/2 \rfloor$ góc cơ sở trên phần tư chu kỳ, với

$$M = \frac{N - 1}{2}. \quad (2)$$

1.2 Nền tảng phần cứng

Vi điều khiển **TMS320F280049C** (họ F28004x) cung cấp:

- CPU C28x 100 MHz, bộ nhớ FLASH on-chip, RAM LS/GS;
- nhiều module ePWM độc lập, mỗi module có **hai** thanh ghi so sánh CMPA và CMPB;
- bộ điều khiển DMA 6 kênh, có thể kích hoạt bởi sự kiện ePWM SOC.

Trong dự án `epwm_svm`, ba pha được phát trên ePWM6 (pha A), ePWM5 (pha B), ePWM3 (pha C); ePWM1 đóng vai trò *master timer* phát ngắt và tín hiệu SOC-A cho DMA. Tham số sóng mang:

$$f_{\text{carr}} = 50 \text{ kHz}, \quad \text{TBPRD} = 2000 \text{ counts}, \quad (3)$$

$$f_1 = \frac{f_{\text{carr}}}{N_{\text{carr}}} = \frac{50 \text{ kHz}}{100} = 500 \text{ Hz}, \quad \Delta\theta_{\text{slot}} = \frac{360^\circ}{N_{\text{carr}}} = 3,6^\circ. \quad (4)$$

2 Vấn đề 1: Giới hạn phần cứng — hai CMP nhưng cần $4M$ sự kiện

2.1 Phát biểu bài toán

Mỗi module ePWM chỉ có **hai** ngưỡng so sánh (CMPA, CMPB) cho mỗi chu kỳ sóng mang. Tuy nhiên, sau khi mở rộng đối xứng từ M góc cơ sở, mỗi pha cần tới

$$N_{\text{sw}} = 4M = 2(N - 1) \quad (5)$$

điểm chuyển mạch trong một chu kỳ cơ bản 360° . Bảng 1 liệt kê quy mô sự kiện.

Nếu gán trực tiếp mỗi sự kiện vào CMPA/CMPB trong ISR, CPU phải tính toán và ghi 12–28 cặp giá trị *mỗi chu kỳ cơ bản*, đồng thời vẫn không thể phát quá **hai** chuyển mạch trên **một** chu kỳ sóng mang — trong khi nhiều chu kỳ mang liên tiếp có thể chứa 0, 1 hoặc 2 sự kiện.

Table 1: Số góc cơ sở M và số sự kiện chuyển mạch $4M$ theo N .

N	M	$4M$	Góc cơ sở (LUT)	Ví dụ $m = 0,50$
7	3	12	3	$\alpha_1, \alpha_2, \alpha_3$
9	4	16	4	
11	5	20	5	
13	6	24	6	
15	7	28	7	

2.2 Nguyên lý giải pháp: bảng lịch theo chu kỳ mang

Thay vì “một CMP cho một góc toàn cục”, hệ thống **phân bổ thời gian theo lưới sóng mang**. Chia chu kỳ cơ bản thành $N_{\text{carr}} = 100$ khe (slot), mỗi khe tương ứng một chu kỳ ePWM và góc quét

$$\Delta\theta = \frac{360^\circ}{N_{\text{carr}}} = 3,6^\circ. \quad (6)$$

Với mỗi góc chuyển mạch toàn cục $\theta_g \in [0^\circ, 360^\circ)$:

$$k = \left\lfloor \frac{\theta_g}{\Delta\theta} \right\rfloor, \quad \theta_{\text{loc}} = \theta_g - k \Delta\theta, \quad (7)$$

$$\text{CMP} = \text{round}\left(\frac{\theta_{\text{loc}}}{\Delta\theta} \text{TBPRD}\right), \quad \text{CMP} < \text{TBPRD}. \quad (8)$$

Mỗi phần tử bảng lịch lưu một cặp:

$$\text{SOPWM_CycleCmp_t} = \{\text{cmpa}, \text{cmpb}\}, \quad (9)$$

trong đó $\text{cmpa} \leq \text{cmpb}$ khi cả hai đều hợp lệ. Giá trị đặc biệt $\text{SOPWM_CMP_OFF} = 0\text{xFFFF}$ (lớn hơn TBPRD) báo hiệu ngưỡng *vô hiệu* — Action Qualifier không kích hoạt tại điểm đó.

2.3 Mở rộng quarter-wave từ LUT

Từ M góc cơ sở $\{\alpha_1, \dots, \alpha_M\}$ (độ, trong phần tư đầu), ta xây dựng $4M$ góc toàn chu kỳ:

$$\begin{aligned} \text{Q1: } \theta_i &= \alpha_i, & i &= 1, \dots, M, \\ \text{Q2: } \theta_{M+i} &= 180^\circ - \alpha_{M+1-i}, \\ \text{Q3: } \theta_{2M+i} &= 180^\circ + \alpha_i, \\ \text{Q4: } \theta_{3M+i} &= 360^\circ - \alpha_{M+1-i}. \end{aligned} \quad (10)$$

Đây chính là công thức được triển khai trong `SOPWM_BuildSchedule()`.

2.4 Đối xứng nửa chu kỳ và ba pha

Để đảm bảo dòng trung tính và đóng chu kỳ sóng leg, hàm `sopwm_halfwave_closure()` ép một chuyển mạch tại khe 180° của từng pha:

$$k_{\text{mid},A} = 50, \quad k_{\text{mid},B} = 83, \quad k_{\text{mid},C} = 17, \quad (11)$$

tương ứng lệch pha 120° và 240° trên lưới 100 khe ($120^\circ/3,6^\circ = 33$ khe). Pha B/C được tạo bằng **xoay vòng** bằng pha A:

$$\text{sched}[\phi][k] = \text{sched}[A][(k + \Delta k_\phi) \bmod N_{\text{carr}}], \quad \Delta k_B = 67, \Delta k_C = 33. \quad (12)$$

Như vậy, **hai thanh ghi CMP** vẫn đủ cho *một chu kỳ mang*, còn toàn bộ $4M$ sự kiện được “trải” lên N_{carr} chu kỳ mang thông qua bảng lịch `sopwm_sched[3][2][100]`.

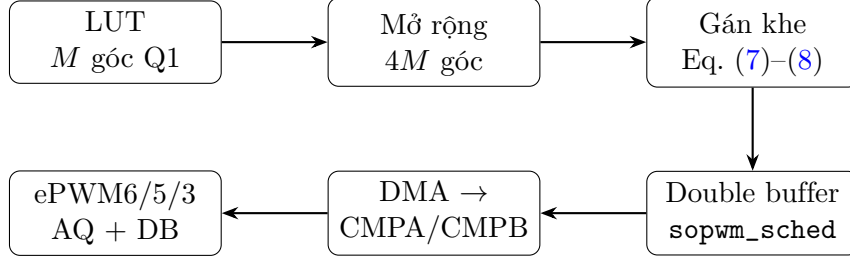


Figure 1: Luồng dữ liệu giải quyết giới hạn hai CMP: LUT → mở rộng $4M$ góc → bảng lịch theo khe → DMA nạp CMP.

3 Vấn đề 2: Tính cần thiết của DMA trong cập nhật góc

Bảng lịch `sopwm_sched` giải quyết mâu thuẫn “nhiều sự kiện – hai CMP”, nhưng vẫn cần cơ chế nạp 600 cặp giá trị CMP vào ba module ePWM mỗi chu kỳ cơ bản. Phần này phân tích vì sao CPU không đáp ứng được và vai trò bắt buộc của DMA.

3.1 Tại sao CPU không đủ

Mỗi chu kỳ sóng mang ($20\mu\text{s}$ tại 50kHz), ePWM cần một cặp giá trị CMPA/CMPB mới. Trong một chu kỳ cơ bản:

$$N_{\text{writes}} = N_{\text{carr}} \times 3 \text{ pha} \times 2 \text{ reg} = 600 \text{ lần ghi 16-bit}. \quad (13)$$

Nếu thực hiện trong ISR ePWM1 (50kHz):

- ISR phải ghi 6 thanh ghi so sánh ($3 \text{ pha} \times 2 \text{ CMP}$) *trước* khi counter vượt ngưỡng — cực kỳ nhạy jitter;
- thời gian ISR tăng tuyến tính theo số pha; dễ xung đột với vòng điều khiển, ADC, truyền thông;
- shadow register chỉ an toàn khi ghi đúng thời điểm ZERO/COMP — DMA đồng bộ phần cứng đảm bảo hơn ghi CPU tùy ý.

3.2 Cấu hình DMA trong dự án

Ba kênh DMA (CH0–CH2) cấu hình qua SysConfig với tham số chung:

$$\text{Burst size} = 2 \quad (\text{CMPA} + \text{CMPB liên tiếp}), \quad (14)$$

$$\text{Transfer size} = N_{\text{carr}} = 100, \quad (15)$$

$$\text{Src wrap size} = 200 = 2 \times 100 \quad (\text{double buffer}), \quad (16)$$

$$\text{Trigger} = \text{DMA_TRIGGER_EPWM1SOCA} @ \text{TBCTR}=\text{ZERO}. \quad (17)$$

Dịch DMA trở tới vùng shadow CMPA (offset EPWM_0_CMPA + 1) của ePWM6, ePWM5, ePWM3. Mỗi lần ZERO:

1. DMA burst: đọc {cmpa, cmpb} từ `sopwm_sched[phi][b][k]`;
2. ghi vào shadow CMPA/CMPB của pha ϕ ;
3. tăng chỉ số nguồn; sau 100 burst thì quay vòng (wrap) trong phạm vi 200 word.

Công thức ánh xạ thời gian — góc trên *một* chu kỳ mang k :

$$\theta_k = k \cdot \Delta\theta + \theta_{\text{loc}}, \quad t_{\text{cmp}} = \frac{\text{CMP}}{\text{TBPRD}} T_{\text{carr}}. \quad (18)$$

Action Qualifier cấu hình TOGGLE tại TBCTR = CMPA/CMPB (up-count), tạo xung SOPWM với dead-band 480 counts ($\approx 2.4 \mu\text{s}$).

3.3 Đồng bộ DMA – ISR – vòng lặp chính

Kiến trúc thời gian thực ba tầng:

1. **DMA (50 kHz):** nạp CMP — tầng *hard real-time*, không tham gia CPU.
2. **ISR ePWM1 (50 kHz):** `SOPWM_schedISR()` đếm k , hoán đổi buffer tại $k = 0$, báo `sopwm_fund_tick`; cập nhật con trỏ DMA khi rollover.
3. **Main loop (500 Hz):** khi `sopwm_fund_tick`, gọi

```
SOPWM_BuildSchedule(sopwm_m_cmd, sopwm_N_cmd, SOPWM_TBPRD);
SOPWM_CommitSchedule();
```

Tầng này có thể cập nhật m từ vòng kín mà không ảnh hưởng jitter PWM.

Quy tắc quan trọng: **cả ba kênh DMA dùng chung trigger EPWM1SOCA** để tránh race — ISR chạy sau DMA burst, cập nhật `DMA_configAddresses()` trước trigger tiếp theo (20 μs sau).

3.4 Chỉ số điều chế và cập nhật góc

Chỉ số điều chế (open-loop hoặc closed-loop):

$$m = \frac{\pi}{2U_{\text{dc}}} \sqrt{U_{\alpha}^2 + U_{\beta}^2}, \quad 0,01 \leq m \leq 1,00. \quad (19)$$

Mỗi chu kỳ cơ bản, m mới $\Rightarrow$ tra LUT $\Rightarrow$ dựng lại toàn bộ bảng 100 khe $\times$ 3 pha. DMA đảm bảo **mỗi thay đổi góc** (dù nhỏ) được phản ánh đúng vào CMP *đúng chu kỳ mang*, điều kiện cần của điều chế đồng bộ.

4 Vấn đề 3: Phân bố bộ nhớ FLASH/RAM

4.1 Phát biểu vấn đề

Triển khai SOPWM trên TMS320F280049C không chỉ cần “có đủ byte” — mỗi loại dữ liệu phải nằm đúng **loại bộ nhớ vật lý** và đúng **section liên kết**, nếu không hệ thống sẽ lỗi im lặng hoặc build thất bại. Đây là bài toán cấu trúc, không phải tối ưu phụ.

(a) **Khối lượng LUT gốc lớn so với RAM LS.** Mỗi cặp (N, m) cần lưu trước $M = (N - 1)/2$ góc chuyển mạch (độ). Với năm mức N và 100 bước m , tổng cộng:

$$S_{\text{LUT,naive}} = 100 \times 4 \times (3 + 4 + 5 + 6 + 7) = 10\,000 \text{ byte.} \quad (20)$$

Nếu đặt toàn bộ LUT vào RAM (như bảng sin/cos tham chiếu trong module SVPWM trước đó), section `.bss/.const` để **tràn RAMLS5** — lỗi linker đã ghi nhận trong `log.md`.

(b) **Bảng lịch runtime không được nằm FLASH.** Sau khi tra LUT, `SOPWM_BuildSchedule()` ghi liên tục vào bảng `sopwm_sched` — nguồn burst của DMA. Dữ liệu này:

- thay đổi mỗi chu kỳ cơ bản (500 Hz);
- phải nằm trong vùng RAM mà **DMA bus master** truy cập được (GS RAM trên F28004x);
- cần **hai bản sao** (double buffer) để CPU ghi buffer inactive trong khi DMA đọc buffer active.

Đặt nhầm vào RAM LS hoặc FLASH sẽ khiến DMA đọc sai, hoặc ghi CMP ra giá trị 0xFFFF.

(c) **Xung đột nhu cầu truy cập.** Cùng một lúc hệ thống cần:

Thành phần	Truy cập	Ràng buộc vùng
LUT gốc	CPU đọc, không đổi	FLASH <code>.const</code>
<code>sopwm_sched</code>	CPU ghi + DMA đọc	RAMGS, section riêng
Stack / biến debug	CPU R/W	RAMLS / <code>.bss</code>
Mã <code>BuildSchedule</code>	CPU thực thi	FLASH <code>.text</code>

(d) **Chi phí bộ nhớ tăng theo tham số thiết kế.** Nếu lưu thẳng $4M$ góc đã mở rộng (thay vì M góc Q1), dung lượng LUT nhân bốn:

$$S_{\text{LUT,full}} = 4 \times S_{\text{LUT,naive}} = 40 \text{ kB.} \quad (21)$$

Tương tự, bảng lịch tăng tuyến tính theo N_{carr} và số pha:

$$S_{\text{sched}} = N_{\phi} \times N_{\text{buf}} \times N_{\text{carr}} \times \text{sizeof}(\text{SOPWM_CycleCmp_t}). \quad (22)$$

Với $N_{\phi} = 3$, $N_{\text{buf}} = 2$, $N_{\text{carr}} = 100$, mỗi phần tử 4 byte:

$$S_{\text{sched}} = 3 \times 2 \times 100 \times 4 = 2400 \text{ byte.} \quad (23)$$

Tóm lại, vấn đề cần giải quyết là: *phân tách dữ liệu tĩnh (LUT) và dữ liệu động (bảng lịch DMA), map đúng section trong linker, đồng thời tối giảm footprint nhờ đối xứng quarter-wave — không tính góc runtime trên CPU.*

Table 2: Vùng bộ nhớ on-chip liên quan SOPWM.

Symbol linker	Địa chỉ gốc	Dung lượng	Vai trò trong SOPWM
FLASH_BANK0_SEC4	0x084000	4 KiB	Section <code>.const</code> — LUT
RAMGS0	0x00C000	8 KiB	Section <code>ramgs0</code> — <code>sopwm_sched</code>
RAMLS5	0x00A800	2 KiB	<code>.bss</code> , <code>.data</code> — biến toàn cục
RAMM1	0x000400	≈1 KiB	Stack

4.2 Cấu hình bộ nhớ trong dự án

4.2.1 Bản đồ vật lý F280049C (trích 28004x_generic_flash_lnk.cmd)

4.2.2 Cấu hình section trong file liên kết

File 28004x_generic_flash_lnk.cmd khai báo hai section **do dự án thêm** (ngoài mặc định C2000Ware):

```
/* PAGE 1 -- MEMORY block */
RAMGS0 : origin = 0x00C000, length = 0x002000 /* 8 KiB */

/* SECTIONS */
.const : > FLASH_BANK0_SEC4, PAGE = 0, ALIGN(4) /* const
float LUT */
ramgs0 : > RAMGS0, PAGE = 1 /* DMA
source table */
ramgs1 : > RAMGS1, PAGE = 1 /* du phong
mo rong */
```

Lý do chọn FLASH_BANK0_SEC4 cho `.const`. Sector 4 KiB đủ chứa ≈10 KiB LUT (chiếm ≈ 24% sector), tách biệt khỏi `.text` (SEC2/3/5) và `.cinit` (SEC1), tránh phải gom mã và hằng số vào cùng sector gây khó mở rộng.

Lý do chọn RAMGS0 cho bảng lịch. Theo TRM F28004x, DMA có thể truy cập Global Shared RAM (GS). `sopwm_sched` chiếm 2400 byte trên 8192 byte RAMGS0 (≈ 29%), còn dư cho biến DMA khác. Section `ramgs1` được khai báo sẵn trên RAMGS1 (8 KiB) cho mở rộng sau này.

4.2.3 Ràng buộc build CCS

- Cấu hình **CPU1_FLASH** (hoặc tương đương) dùng 28004x_generic_flash_lnk.cmd.
- `Flash_initModule(..., DEVICE_FLASH_WAITSTATES)` trong `device.c`: `WAITSTATES = 4 @ 100 MHz` — LUT đọc từ FLASH vẫn đủ nhanh vì `BuildSchedule` chỉ chạy 500 Hz, không phải 50 kHz.
- `DMA srcWrapSize = 200` trong `SysConfig` khớp $2 \times N_{\text{carr}}$ word nguồn (double buffer liên tiếp trong RAM).

4.3 Đóng góp của mã nguồn

Bảng 3 tóm tắt cách code giải quyết vấn đề phân bố bộ nhớ.

Table 3: Đóng góp mã nguồn cho bài toán bộ nhớ.

Thành phần	File	Đóng góp
LUT tĩnh	sopwm.c	Năm mảng <code>const float SOPWM_LUT_N*</code> (≈ 10 kB) tự động vào <code>.const/FLASH</code> ; không dùng <code>malloc</code> , không copy sang RAM lúc khởi động.
Tra cứu $O(1)$	sopwm.h	<code>SOPWM_GetAngles()</code> <code>static inline</code> : map $m \rightarrow i_m = \text{round}(100m - 1)$, <code>switch(N)</code> — thời gian xác định, không stack frame.
Placement DMA	sopwm.c	<code>#pragma DATA_SECTION(sopwm_sched, "ramgs0")</code> buộc linker đặt bảng vào RAMGS0; comment ghi rõ yêu cầu linker.
Kiểu dữ liệu gọn	sopwm.h	<code>SOPWM_CycleCmp_t</code> = <code>2×uint16_t</code> (4 byte/slot), không dùng <code>float</code> trong bảng DMA — giảm S_{sched} và khớp burst DMA (2 word).
Double buffer	sopwm.c/h	<code>sopwm_sched[3][2][SOPWM_N_CARR];</code> <code>sopwm_sched_active,</code> <code>sopwm_sched_next;</code> <code>BuildSchedule</code> ghi <code>[next]</code> , ISR hoán đổi — tránh ghi đè vùng DMA đang đọc.
Tiết kiệm FLASH	sopwm.c	<code>SOPWM_BuildSchedule()</code> : chỉ đọc M góc Q1 từ LUT, mở rộng $4M$ góc bằng (10) trên stack (<code>all_angles[28]</code>) — không lưu $4M$ góc/cấp m trong FLASH.
Linker	28004x_generic_flash.ld	<code>Merge</code> <code>const</code> sang FLASH, <code>ramgs0</code> sang RAMGS0; tách LUT khỏi RAMLS tránh tràn như SVPWM.

4.3.1 Minh hoạ placement trong sopwm.c

LUT — compiler đặt vào .const (FLASH):

```
const float SOPWM_LUT_N7[SOPWM_M_COUNT][3] = {
    /* m=0.01 */ {60.118f, 80.177f, 80.364f},
    /* ... 100 rows ... */
};
```

Bảng lịch — placement explicit sang RAMGS:

```
#pragma DATA_SECTION(sopwm_sched, "ramgs0")
SOPWM_CycleCmp_t sopwm_sched[3][2][SOPWM_N_CARR];
```

BuildSchedule — đọc FLASH, ghi RAMGS (buffer inactive):

```
void SOPWM_BuildSchedule(float m, uint16_t N, uint16_t tbprd)
{
    float base_deg[7];
    float all_angles[28]; /* 4*7 max -- stack, không FLASH */
    n_base = SOPWM_GetAngles(m, N, base_deg); /* doc LUT FLASH */
    /* ... mo rong quarter-wave vao all_angles[] ... */
    sopwm_bin_lut(all_angles, total, tbprd,
                  sopwm_sched[0][sopwm_sched_next]); /* ghi RAMGS */
}
```

4.4 Sơ đồ phân vùng logic

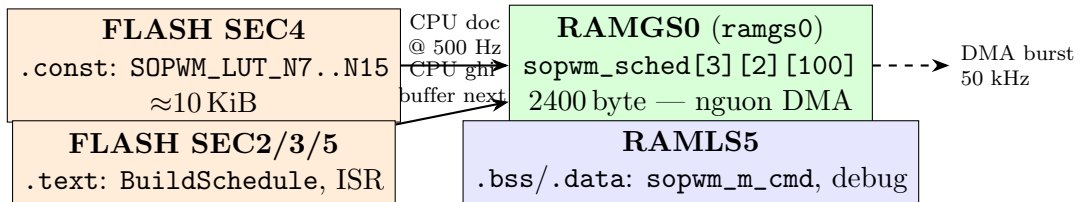


Figure 2: Phân vùng logic: LUT tính trên FLASH; bảng lịch động trên RAMGS; CPU là cầu nối, DMA đọc trực tiếp RAMGS.

4.5 Chi tiết dung lượng LUT

Chỉ số tra cứu (code):

$$i_m = \text{round}(100m - 1), \quad i_m \in [0, 99]. \quad (24)$$

4.6 Kết quả và hệ quả thiết kế

1. **Linker thành công** với LUT trên FLASH và bảng lịch trên RAMGS — không tràn RAMLS như phiên bản SVPWM.
2. **DMA hoạt động ổn định** vì nguồn nằm đúng GS RAM; srcWrapSize=200 khớp layout [3 pha][2 buf][100] khi mỗi kênh DMA phục vụ một pha.

Table 4: Dung lượng từng bảng SOPWM_LUT_N*.

Bảng	Kích thước	Phần tử	Bytes
SOPWM_LUT_N7	100×3	300	1200
SOPWM_LUT_N9	100×4	400	1600
SOPWM_LUT_N11	100×5	500	2000
SOPWM_LUT_N13	100×6	600	2400
SOPWM_LUT_N15	100×7	700	2800
Tổng		2500 float	10000

3. **Footprint LUT giảm** $\times 4$ nhờ lưu M góc Q1 + mở rộng runtime; footprint bảng lịch giảm nhờ uint16_t thay vì float cho CMP.
4. **Chi phí mở rộng có thể dự báo:** tăng N_{carr} từ 100 lên 200 sẽ đẩy S_{sched} lên 4800 byte; tăng số mức m từ 100 lên 1000 sẽ nhân mười dung lượng FLASH LUT.

Kiểm tra sau build: trong CCS, cửa sổ Memory Browser xác nhận địa chỉ `sopwm_sched` thuộc vùng 0x00C000–0x00DFFF; map file liệt kê SOPWM_LUT_N* trong section `.const`.

5 Công thức tổng hợp và ví dụ số

5.1 Ví dụ: $N = 7$, $m = 0,50$

Từ LUT: $M = 3$, góc cơ sở (độ) $\alpha_1 \approx 66,45$, $\alpha_2 \approx 76,52$, $\alpha_3 \approx 85,21$. Mở rộng (10) cho 12 góc. Góc $\theta \approx 85,21^\circ$ rơi vào

$$k = \lfloor 85,21/3,6 \rfloor = 23, \quad \theta_{\text{loc}} = 85,21 - 23 \times 3,6 = 2,41^\circ,$$

$$\text{CMP} = \text{round}\left(\frac{2,41}{3,6} \times 2000\right) \approx 1339.$$

Giá trị này khớp biến debug `dbg_cmpa_snapshot` ≈ 1339 tại `sopwm_cycle_idx = 24` trong `main.c`.

5.2 Điều kiện tính đúng của SOPWM đồng bộ

Để waveform SOPWM hợp lệ cần đồng thời:

$$(i) \quad \sum_k \text{số toggle trên leg} \equiv 0 \pmod{2} \quad (\text{đóng chu kỳ}), \quad (25)$$

$$(ii) \quad \text{toggle bắt buộc tại } 180^\circ \text{ mỗi pha}, \quad (26)$$

$$(iii) \quad \text{CMPA/CMPB được nạp trước mỗi chu kỳ mang (DMA @ ZERO)}. \quad (27)$$

Hàm `sopwm_halfwave_closure()` xử lý (i)–(ii); DMA xử lý (iii).

6 Kết luận

Triển khai SOPWM trên TMS320F280049C đã giải quyết có hệ thống ba thách thức:

1. **Giới hạn hai CMP:** chuyển từ “một góc – một CMP” sang **bảng lịch theo chu kỳ mang** với tối đa hai sự kiện/khe, mở rộng $4M$ góc bằng đối xứng quarter-wave.
2. **DMA:** nạp 600 giá trị CMP/chu kỳ cơ bản với jitter cực thấp, tách tính toán SOPWM (500 Hz) khỏi nạp phần cứng (50 kHz).
3. **Bộ nhớ LUT:** lưu M góc/ m trong FLASH (~ 10 KiB), bảng runtime DMA trong RAMGS (2.4 KiB), double buffer tránh ghi/đọc đồng thời.

Hướng mở rộng: nội suy LUT theo m , tích hợp vòng kín trực tiếp trong ISR thông qua CLA, hoặc mở rộng N_{carr} để giảm $\Delta\theta$ và tăng độ phân giải góc — với chi phí RAM bảng lịch tăng tuyến tính.

A Tham chiếu mã nguồn

- `pwm/sopwm/sopwm.c`, `sopwm.h` — LUT, BuildSchedule, DMA table.
- `main.c` — khởi tạo DMA, ISR, vòng lặp cơ bản.
- `epwm_sync_svm.syscfg` — cấu hình ePWM/DMA.
- `28004x_generic_flash_lnk.cmd` — map `.const`, `ramgs0`.

References

- [1] D. G. Holmes, T. A. Lipo, *Pulse Width Modulation for Power Converters*, Wiley, 2003.
- [2] Texas Instruments, *TMS320F28004x Technical Reference Manual*, ePWM & DMA chapters.
- [3] B. Ozpineci, “Selective Harmonic Elimination PWM”, TI Application Reports.